

Assignment 1 — Introduction to Web Development

1. Difference between Frontend, Backend, and Full-Stack Development

Frontend development is the part of web development concerned with what the user sees and interacts with directly in the browser — layout, design, buttons, forms, and animations. It is built using HTML, CSS, and JavaScript, often with frameworks like React, Angular, or Vue. Example: designing the homepage of Amazon that shows product images, search bar, and navigation menu.

Backend development is the server-side part of an application that handles business logic, database operations, authentication, and server configuration. It is built using languages/frameworks such as Node.js, Python (Django/Flask), Java (Spring), or PHP. Example: the system on Amazon's servers that checks stock availability and processes a payment when a user places an order.

Full-stack development combines both frontend and backend skills, allowing a developer to build a complete, end-to-end application. Example: a full-stack developer could build both the shopping cart interface (frontend) and the order-processing API (backend) for an e-commerce site.

2. Diagram: Client-Server Model in Web Architecture

The client-server model describes how a user's device (client) communicates with a remote computer (server) over a network to request and receive resources such as web pages, data, or files.

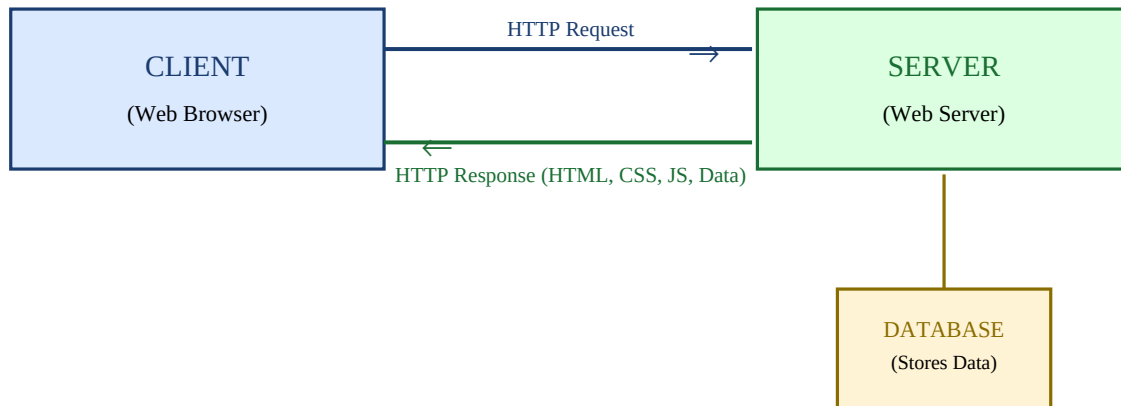


Figure 1: The client sends an HTTP request to the server; the server processes it (fetching data from the database if needed) and sends back an HTTP response.

3. How a Browser Requests and Displays a Web Page

- 1 The user types a URL (e.g., `www.example.com`) into the browser's address bar and presses Enter.
- 2 The browser sends the domain name to a DNS (Domain Name System) server, which resolves it into an IP address.
- 3 The browser opens a TCP connection to the server at that IP address and sends an HTTP/HTTPS request for the page.

- 4 The web server receives the request, processes it (possibly querying a database or running backend code), and sends back an HTTP response containing HTML, CSS, JavaScript, and other assets.
- 5 The browser's rendering engine parses the HTML to build the DOM (Document Object Model) and the CSS to build the CSSOM, then combines them into a render tree.
- 6 JavaScript is executed, and the browser paints pixels on the screen to display the fully rendered web page to the user.

4. Tools Required to Set Up a Web Development Environment

Tool	Purpose
Code Editor / IDE (e.g., VS Code)	Writing, editing, and organizing source code with features like syntax highlighting, extensions, and debugging.
Web Browser (e.g., Chrome, Firefox)	Running and testing the web application; browser DevTools help inspect elements, debug JavaScript, and monitor network requests.
Version Control System (Git)	Tracking changes to code, collaborating with other developers, and maintaining a history of the project.
Code Hosting Platform (GitHub/GitLab)	Storing repositories online, enabling collaboration, backups, and deployment pipelines.
Package Manager (npm/yarn/pip)	Installing, updating, and managing external libraries and dependencies.
Local/Development Server (Live Server, XAMPP, Node.js server)	Running the website locally to preview changes before deploying it live.
Browser Developer Tools	Inspecting HTML/CSS, debugging JavaScript, and checking performance and network activity.
Terminal / Command Line	Running commands to install packages, start servers, and manage version control.

5. What is a Web Server?

A web server is software (often running on dedicated hardware) that stores, processes, and delivers web pages and other resources to clients over the internet using HTTP/HTTPS. When a browser requests a page, the web server locates the requested file or generates the content dynamically and sends it back as an HTTP response.

Commonly used web servers include:

- Apache HTTP Server — one of the oldest and most widely used open-source web servers.
- Nginx — known for high performance, load balancing, and handling many concurrent connections.
- Microsoft IIS (Internet Information Services) — used mainly for Windows-based hosting.
- Node.js (with Express) — allows developers to build a custom web server using JavaScript.
- LiteSpeed — a high-performance commercial alternative to Apache.

6. Roles: Frontend Developer, Backend Developer, Database Administrator

Role	Responsibilities
Frontend Developer	Designs and implements the user interface using HTML, CSS, and JavaScript; ensures the site is responsive, accessible, and visually consistent; works closely with UI/UX designers.
Backend Developer	Builds and maintains the server, application logic, and APIs; handles authentication, business logic, and integration with the database; ensures performance and security of server-side code.
Database Administrator (DBA)	Designs, implements, and maintains the database; manages data storage, backups, security, and performance tuning; ensures data integrity and handles migrations.

7. Install and Configure VS Code for HTML, CSS, and JavaScript

- 1 Download VS Code from the official website (code.visualstudio.com) for your operating system (Windows/macOS/Linux) and run the installer.
- 2 Open VS Code and go to the Extensions panel (Ctrl+Shift+X). Install helpful extensions such as: Live Server (to preview HTML pages with auto-refresh), Prettier (code formatter), ESLint (JavaScript linting), and Auto Rename Tag / Auto Close Tag (for HTML editing convenience).
- 3 Create a project folder, e.g. my-website, and open it in VS Code via File > Open Folder.
- 4 Create index.html, style.css, and script.js files inside the folder, and link the CSS and JS files inside the HTML using the link and script tags.
- 5 Right-click index.html and choose 'Open with Live Server' to preview the page in the browser with live reloading as you edit.

Note: Since this is a text-based submission, an actual setup screenshot could not be captured here. Please insert a screenshot of your own VS Code window (showing the file explorer, an open HTML file, and the installed extensions) in this section before final submission.

8. Static vs Dynamic Websites

A **static website** displays the same fixed content to every visitor. It is built with HTML and CSS files that are sent to the browser exactly as stored on the server, with no server-side processing of data for each request. It is fast, simple, and cheap to host. **Example:** a simple personal portfolio website that just shows a resume and contact information, which stays the same for every visitor.

A **dynamic website** generates content on the fly, often based on user input, database queries, or session data, so different users can see different content. It requires server-side programming (e.g., PHP, Node.js, Python) and usually a database. **Example:** an online store like Amazon, where the homepage shows personalized product recommendations and login-based account information for each user.

9. Five Web Browsers and Their Rendering Engines

Browser	Rendering Engine	Notes
Google Chrome	Blink	Blink is a fork of WebKit, developed by Google; also powers Chromium-based browsers.

Mozilla Firefox	Gecko	Gecko is developed by Mozilla and is known for strong standards compliance.
Apple Safari	WebKit	WebKit was originally developed by Apple (derived from KHTML); used on macOS and iOS.
Microsoft Edge	Blink (Chromium-based)	Modern Edge switched from EdgeHTML to Chromium's Blink engine in 2020.
Opera	Blink (Chromium-based)	Opera also moved to the Blink engine after being based on its own Presto engine earlier.

Rendering engines differ mainly in how strictly and quickly they interpret HTML/CSS standards, their JavaScript engine pairing (e.g., Blink pairs with V8, Gecko pairs with SpiderMonkey, WebKit pairs with JavaScriptCore), and subtle differences in layout, font rendering, and support for new web features — which is why developers test websites across multiple browsers.

10. Diagram: Basic Web Architecture Flow

The diagram below shows the full flow of a request from the client through the server, APIs, and database.

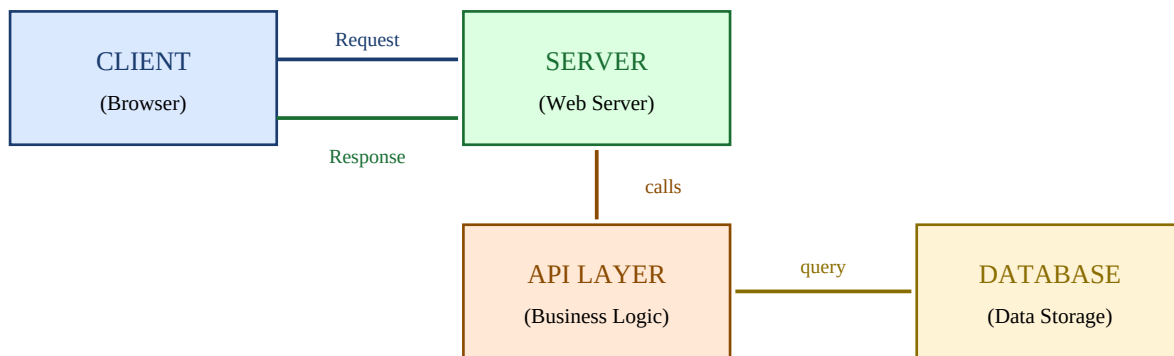


Figure 2: The client sends a request to the server; the server routes it through the API layer, which queries the database and returns data back up the chain to the client.